Database Management, Fall 2025 (115-1)
Homework 1

Instructor: Ling-Chieh Kung

Department of Information Management
National Taiwan University

繳交作業時，請至 NTU COOL 下載本作業題目的地方上傳一個 PDF 檔。在生成這個 PDF 檔時，可以用打字的也可以用手寫的，但不管怎樣，請務必注意繳交的文件的專業程度（通常透過排版、文字圖片表格方程式的清晰程度、用字遣詞等面向呈現），如果專業程度不夠，也會被酌量扣分。每位學生都要上傳自己寫的解答。不接受紙本繳交；不接受遲交。可以用中文或英文作答。這份作業的截止時間是 ***9 月 28 日晚上 23:59:59***，遲交在 12 小時內者扣 10 分、在 12 到 24 小時內者扣 20 分、超過 24 小時的這份作業將得不到分數。

本次作業滿分 120 分。學生可以兩人一組，若是如此則只要一個人上傳即可，但請在文件首頁寫明兩人的學號與姓名。有兩人的組若拿到超過 100 分，本次作業就以那個分數計；只有一人的組若拿到超過 100 分，則以 100 分計。一組內若有兩人，兩人會得到同樣的分數。結算學期成績時，作業這一項的分數就是要計分的五次作業分數加總除以五（亦即可以超過 100）。

相關規定與提醒

1. 關於上網查詢與 AI 工具：

   任何一份作業都可以被用任何方式完成，包括上網搜尋和使用各種 AI 工具。如果你想用，請留意以下幾件事。首先，抄襲還是不被允許的，如果我們發現抄襲（包括抄襲網路上的答案，或是抄襲同學的答案），都還是會給予嚴厲的懲罰（視情節輕重而定，通常是該份作業算零分，或者不予通過這門課）。只要沒有抄襲，那我們就只根據你交上來的答案的品質給分，不論你是自己想出來的，還是有利用 AI 工具。如果某甲善用了 AI 工具後寫得很好，某乙自己努力寫但寫得不好，那某甲會得到比較高分。其次，如大家所知，AI 工具給的答案可能會錯，也可能不合適。使用 AI 工具是學生的自由，但確認 AI 工具的答案是否合適、是否需要調整則是學生的任務。請務必自行確認答案的正確性與合適性。最後還是想提醒大家，學到多少東西都是自己的，如果一時困難用 AI 工具度過難關那是無妨，但之後建議還是花時間把東西學起來，對自己比較好。

2. 關於「專業」：

在我們這門課及許多課程中，都需要繳交各種報告。一份報告如果要達成他的效果（例如成功募資、完成溝通的任務等等），除了需要好的內容，也需要「專業」，而顯得專業通常需要「長得好看」以及「看起來用心」，這在沒有標準答案的任務上更是如此。有鑑於此，在這門課的作業和專案，我們都會要求報告的格式和美觀，並且納入評分標準。為此，我們提供報告格式參考指南「DB_reportFormatGuideline.pdf」，上面列舉了一份格式良好的報告的最低標準。在作業一我們會請助教就違反參考指南的地方標出來讓大家知道（我們理解那份指南並不是最完美的，但如果完全沒有標準，同學們容易無所適從，所以我們還是設計一份當標準），但不會扣分，只是提醒大家；從作業二起就會有部分分數是報告專業度分數。之所以要求這些不是想要找大家麻煩，而是大家離出社會也不是太遠了，確實應該要開始被要求報告的可讀性和易讀性，所以我們願意花一些時間要求大家，但不會刁難大家，也請大家理解和盡力嘗試了。

3. 關於「批改」：

   如課程大綱所述，為了不要累死助教，每次作業可能只有部分題目會被批改和給予回饋，但每一題的參考解答都會在作業截止後公佈。如果有一題沒被批改，那所有有寫那一題的學生都會得到那一題的全部分數，但沒寫或期限前沒交作業的自然就不會拿到那一題的分數。最後，請注意是「可能」，換言之也有可能是所有題目都被批改。

題目

1. (10 points) In this assignment, 10 points will be awarded based on the professionalism of the submitted report. Or you may say that no more than 10 points will be deducted for poor formatting or visual presentation. The remaining points will be awarded based on the correctness and appropriateness of your answers to the following questions.

2. (20 points) A league has several baseball teams. A baseball team owns several players. For each team, we know its name and home location (Taipei, Kaohsiung, etc.). For each player, we know her/his name, jersey number, position (pitcher, catcher, infielder, outfielder, etc.), and annual salary. Some players can play multiple positions (e.g., one may be able to be an outfielder or a catcher). A player cannot join more than one team. Each game is played by two teams. For each game, we know its location, date, home-team score, and away-team score.

To make this problem easier, let's assume we only record current players (i.e., retired players will be removed from our database). Moreover, we only record the current situation (i.e., if a player moves from team A to team B, we record only the fact that she/he is in team B).

Please draw an entity-relationship diagram (ER diagram) that can describe this league.

3. (20 points) Continue from Problem 2. Using the notation and graphical conventions introduced in class (such as one box for each attribute, one row of boxes for each table, underlining to indicate a primary key, and arrows to indicate foreign keys), create a relational schema diagram for the baseball team example. For any details that are not explicitly specified, you may make your own reasonable decisions. For example, if an entity type has two key attributes and either could serve as its primary key, you may choose which one to use as the primary key.

4. (40 points; 5 points for each subproblem) A company stores information about its employees, departments, projects, and related entities in the `COMPANY` database. Its relational schema diagram is shown in Figure 1. The tables and attributes are described below:

   - For each employee (`EMPLOYEE`) hired by the company, the database records the employee's name, including the first name (`Fname`), middle initial (`Minit`), and last name (`Lname`), as well as the employee's social security number (SSN, which is similar to a national identification number in Taiwan), address (`Address`), salary (`Salary`), sex (`Sex`), and date of birth (`Bdate`).
   - The company has multiple departments (`DEPARTMENT`). Each department has a unique name (`Dname`) and a unique number (`Dnumber`), and is managed by a particular employee. A department may have offices at multiple locations (`Dlocation`). Each employee belongs to exactly one department.
   - Each department manages multiple projects (`PROJECT`). Each project has a unique name (`Pname`), a unique number (`Pnumber`), and a single location (`Plocation`), and is managed by one department. Each employee may work on multiple projects, which are not necessarily managed by the department to which the employee belongs. The database records the number of hours each employee works on each project per week. It also records the SSN of each employee's immediate supervisor (`Super_ssn`); this attribute is `NULL` if the employee has no immediate supervisor.

- For the purpose of providing insurance coverage for employees' dependents (`DEPENDENT`), the database records each dependent's name (`Dependent_name`), sex (`Sex`), birthday (`Bdate`), and relationship to the employee (`Relationship`).

Because the attribute names are self-explanatory and the schema is designed in a natural manner, no further explanation is provided.

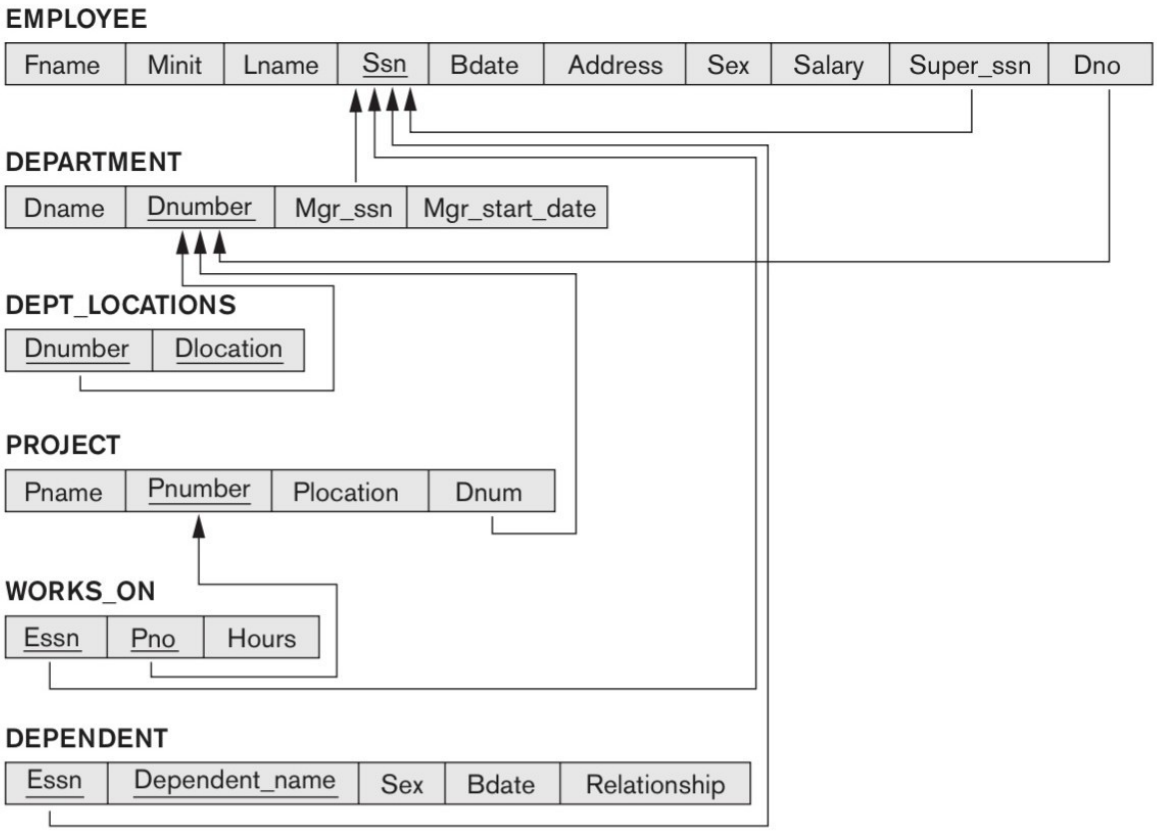


圖 1: Relational schema diagram of the `COMPANY` database

To examine the sample data, you may create an empty database and execute the SQL script, COMPANY_createTableInsertData.sql, to create the tables and insert a set of data. Alternatively, you may restore the attached PostgreSQL backup file, COMPANY_database.backup, which will produce the same tables and data. Using either way, you may obtain information about primary keys, foreign keys, uniqueness constraints, `NOT NULL` constraints, default values, and other relevant settings, but viewing them directly from the script file is easier.

For each of the following query tasks, write one appropriate SQL statement using standard SQL syntax that can be executed correctly in any relational DBMS. When

joining tables, always use `JOIN` and `ON`. Do not specify join conditions in the `WHERE` clause.

(a) Find all female employees in the company whose salary is at least 25,000. Display their SSNs, first names, last names, and salaries. Sort the results by salary in descending order. If two or more employees have the same salary, sort them by last name in ascending order.

(b) Find all employees who were born before January 1, 1965, and whose last names end with the letter `e`. Display their first names, last names, dates of birth, and addresses.

(c) Find all projects managed by a department whose manager was appointed on or after January 1, 1980. Display the project number, project name, project location, department name, and the SSN of the department manager.

(d) For each project, calculate the number of employees who participate in the project and the total number of hours worked on the project per week. Each row of the query result should represent one project and contain the project number, project name, number of participating employees, and total weekly working hours. Do not include projects with fewer than two participating employees. If any employee's weekly working hours for a project are `NULL`, ignore that value when calculating the total.

(e) Find all employees who work on at least one project managed by a department other than the department to which they belong. Each employee should appear only once in the query result. Display the employee's SSN, first name, last name, and the name of the department to which the employee belongs.

(f) For every employee who has at least one direct report, calculate the average salary of the employee's first-level direct reports. Include only direct reports, not employees further down the organizational hierarchy. Display the supervisor's SSN, first name, last name, number of direct reports, and the average salary of those direct reports. Sort the results by the number of direct reports in descending order and then by the supervisor's SSN in ascending order.

(g) For each department, calculate the number of employees who belong to the department and their average salary. Each row of the query result should represent one department and contain the department number, department name, number of employees, and average salary. Include only departments

with at least three employees. Sort the results by average salary in descending order.

(h) For each employee who participates in at least two projects, calculate the number of projects in which the employee participates and the total number of hours the employee works per week. Each row of the query result should contain the employee's SSN, first name, last name, number of projects, and total weekly working hours. If the weekly working hours for any project are `NULL`, ignore that value when calculating the total. Sort the results by the number of projects in descending order and then by SSN in ascending order.

5. (10 points) Following Problem 4, create a data dictionary for this relational schema of `COMPANY` using the format introduced in class. In the process, specify each attribute's data type and possible values as well as constraints such as uniqueness and `NOT NULL`. You should also specify all primary keys and foreign keys. Moreover, please design appropriate referential triggers.

6. (20 points) During the 10-minute demo time:

   (a) (10 points) the TAs will randomly choose one or two subproblems from Problem 4 and use some more data sets to test your SQL statements.

   (b) (5 points) Moreover, they will ask you to explain the rationale behind your SQL statements.

   (c) (5 points) Finally, if you did well in the first two parts and still have time to get here, the TAs will give you an extended query task and ask you to write a new SQL statement.